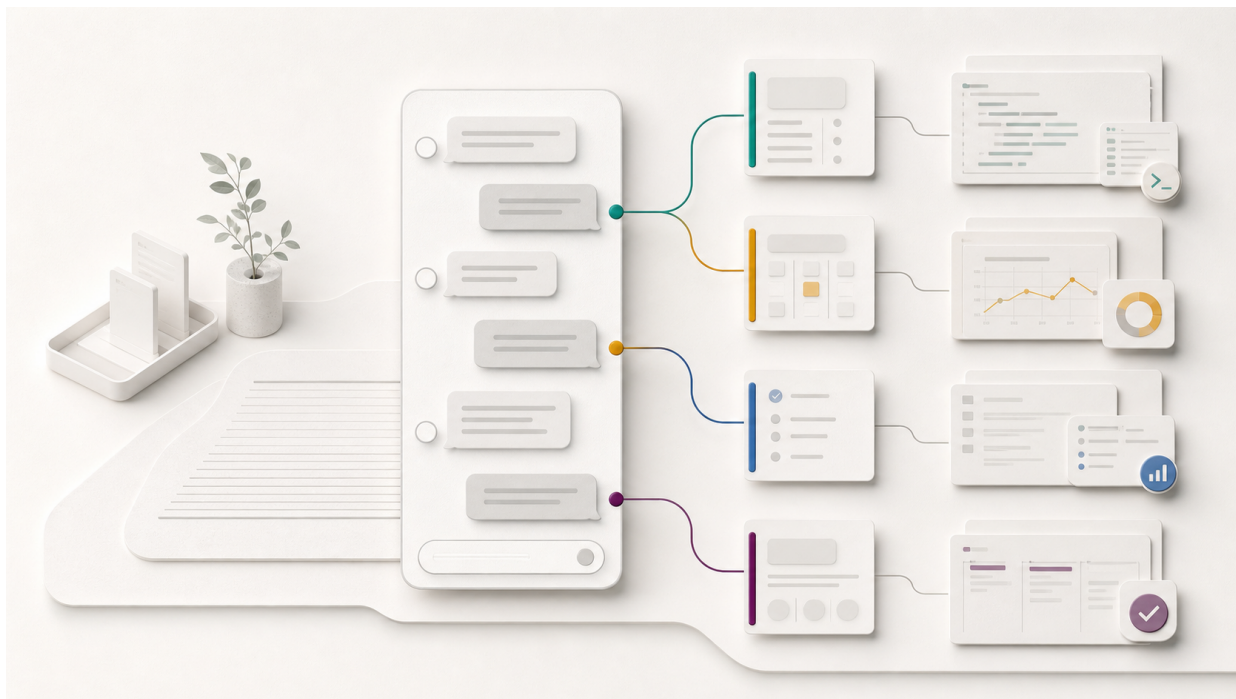


一个入口，但不是一个超级会话

Maka Unified Session 实验说明



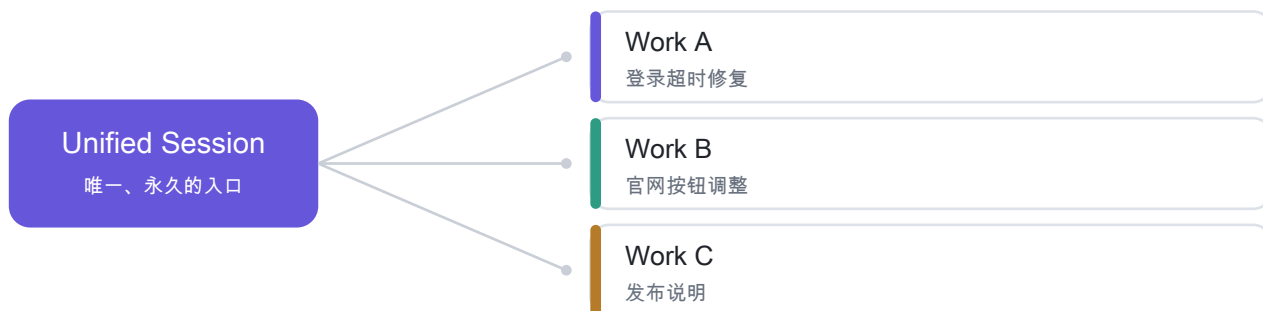
用户始终面对同一个入口，在一个地方了解并推动所有工作；Session 退到幕后，但每项工作的上下文、权限与可恢复性仍然完整保留。

这是一个实验性功能。它不是把所有对话塞进一个超长上下文，而是让 Maka 成为用户与多项工作之间的协调层。

我们真正想减少的，是心智负担

当一个人同时推进多个项目时，他通常必须记住：工作在哪个 Project、对应哪个 Session、上次停在哪里，以及下一句话应该发到哪里。对于不熟悉开发工具的用户，这些都不是工作本身，却会持续占用注意力。

Unified Session 把这种路由负担交给系统。用户只表达意图，Maka 负责识别他正在谈论哪项工作，并把输入绑定到正确的 Session。



对用户来说，入口始终是同一个

- 想知道情况时，直接问：现在手头有哪些工作？
- 想了解进度时，直接问：登录问题处理到哪里了？
- 想继续推进时，直接说：继续刚才那项工作。
- 想同时处理多件事时，在同一个地方安排、确认和跟进。

Session 仍然存在，但它成为承载上下文、权限和执行状态的基础设施，而不是用户每天必须维护的信息架构。

实际运行：仍然像一个普通 Session



真实 Maka 运行界面：Unified 保持普通 Session 的消息流和 Composer；侧栏默认收起，但没有被移除。

Unified 不应该在视觉上成为一个陌生的超级控制台。它仍然使用普通 Session 的布局、消息宽度、气泡与输入区域。用户可以像进入其他 Session 一样进入它，也可以随时打开侧栏，回到任何具体工作。

它与普通 Session 的差异，只保留在确实需要被感知的地方：

- 气泡上方以小字号显示 Project / Work。
- 同一 Work 的问与答使用同色侧边标记。
- 路由完成前保持普通颜色，避免系统过早制造确定感。
- 运行中、完成和需要确认等状态以低噪音事件呈现。

一句输入，三种结果

1. 继续讨论

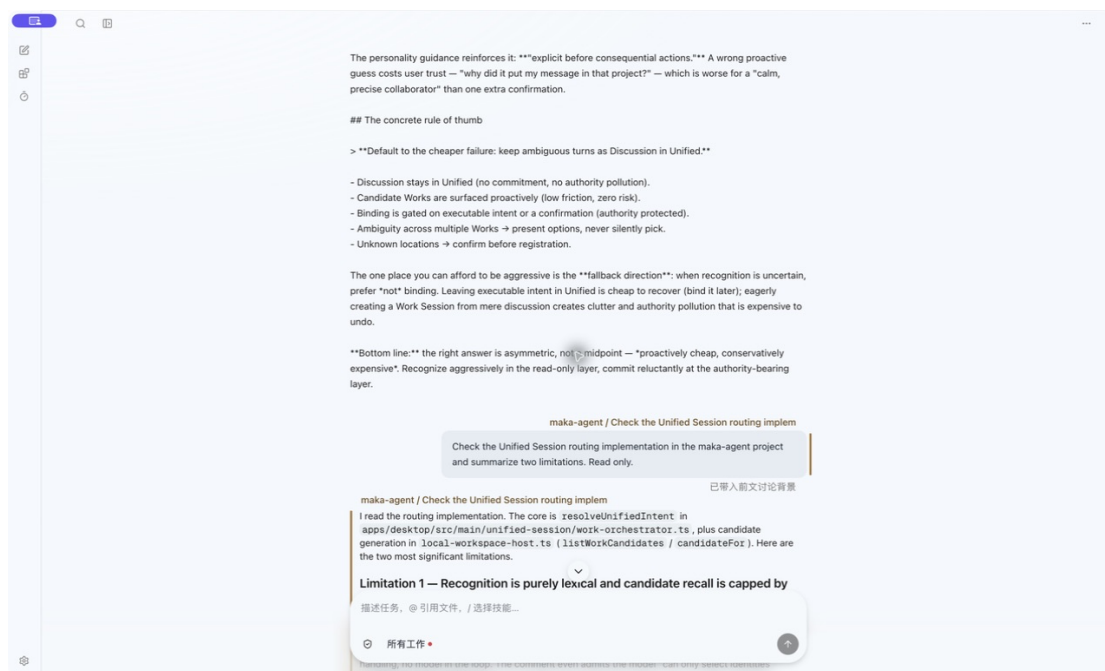
没有明确执行意图时留在 Discussion，不急创建 Work。

2. 自动绑定

目标明确时继续已有 Work，或创建一个普通 Session。

3. 先确认

候选接近或可能产生副作用时，让用户选择目标。



真实 Moka 运行界面：识别完成后显示 Project / Work；中性气泡只通过同色侧边标记提示归属。

这里的关键不是让模型自由猜测。候选 Work 由现有筛选机制提供，语义判断只能在这些候选中选择；若置信度或领先幅度不够，就退回确认，而不是静默错绑。

不确定时，让用户做一次轻量决定



真实 Maka 运行界面：用户可以继续已有 Work，也可以在对应 Project 中创建新 Work。

当目标有歧义，尤其下一步可能修改文件或产生外部副作用时，系统不应该用一个看似聪明的猜测替用户承担风险。确认卡片把决策限制为几个可理解的选项，并保留原工作的执行事实。

- 已经发生的修改不会被假装撤销。
- 用户可以在 Unified 中更改目标 Session 的权限模式。
- 确认后继续原有的 Graph 和依赖关系，不创造另一套执行机制。

无缝，不等于没有边界

Unified 的职责是理解、协调和投影。真正执行工作的仍然是普通 Session；因此每项 Work 的历史、权限、Project、工作目录、工具调用、运行状态、中断恢复与删除语义仍然独立。

无缝来自用户不必管理路由，而不是来自抹掉工作之间的边界。

为什么它对普通用户更重要

开发者往往可以理解 workspace、branch、session 和 runtime 的区别，也更能容忍频繁切换。但对大多数用户来说，这些概念只是软件暴露出来的内部结构。Unified 让产品语言重新接近用户语言：项目、事情、进度、下一步。

为什么它适合移动端

移动端空间有限，层层进入 Project 和 Session 的导航成本更高。一个稳定入口加上可追溯的独立工作块，更符合移动设备上短时、频繁、跨任务的使用方式。用户可以随时问进度、补充一句指令，或处理一张确认卡片。

这个实验要验证什么

- 用户能否在不理解 Session 结构的情况下稳定推进多项工作。
- 系统能否在高频场景中正确识别指代，并在高风险场景中克制地追问。
- 一个统一入口能否与普通 Session 共存，而不牺牲权限、恢复和可解释性。

目标不是消灭 Session，而是让 Session 在不需要被用户操心的时候，安静地退到幕后。